renAIssance: Printed OCR

Organization: HumanAI Foundation **Applicant:** Raymond Frimpong Amoateng **Email:** raymondamoateng@gmail.com **GitHub:** https://github.com/Magma4
Project: RenAIssance: Improving OCR for Early Modern Printed Spanish Sources

---

Synopsis

Most OCR tools were built for modern text. Point one at a page of 17th-century printed Spanish prose and it quietly falls apart. This project is about closing that gap. I want to build a pipeline that handles the specific failure modes of early modern typefaces—the long-s that looks like an f, the near-identical capital I and lowercase l, the abbreviation marks that no modern training corpus has ever seen—and actually produces usable output for researchers who need to work with these sources computationally. The approach combines a custom CNN-RNN recognizer trained with frequency-weighted CTC loss, a lexicon-constrained beam search decoder, and a Gemini LLM post-correction step. The goal is to get Character Error Rate low enough that historians can actually do something useful with the results.

Benefits to Community

If this works, the payoff is real. There are large collections of 17th-century Spanish legal and religious documents sitting in archives that are effectively invisible to computational analysis because no one can search them, link them, or run any kind of text analysis on them. The OCR output is just too noisy. A pipeline that can hold CER below 0.3 on clean pages—which I think is achievable with proper domain-specific fine-tuning—changes that. Documents that researchers currently have to transcribe by hand become machine-readable. Text corpora that could not be assembled at all become possible. And because the whole thing will be open source and built on standard tooling, it's not a one-off research prototype: other groups working on similar historical languages can adapt and extend it.

Deliverables

Right now, the main thing blocking useful CRNN training is line segmentation. When you feed a full page through CTC, the ground truth text is too long relative to the output sequence length and most batches get zeroed out by `zero_infinity=True`. The model does not converge. Fixing this means moving

to per-line crops with per-line labels, which means the line detector needs to actually work first.

Phase 1 — Data and line-level training (Weeks 1 to 4)

- A lightweight trainable line detector (U-Net or YOLO-based) to replace the current projection-profile heuristic. [**Required**]
- Ground truth expansion: the current transcription file has 19 unassigned pages marked TODO. Getting even half of those covered meaningfully increases training data. [**Required**]
- End-to-end CRNN training at the line level with healthy loss curves. [**Required**]

Phase 2 — Model improvements (Weeks 5 to 8)

- Augmentation experiments: synthetic degradation, rotation, ink bleeding. [**Optional**]
- Self-supervised pre-training on unlabelled page crops before fine-tuning on ground truth. [**Optional**]
- Explore replacing the BiLSTM head with a lightweight cross-attention layer as a CRNN variant. [**Optional**]

Phase 3 — Evaluation and delivery (Weeks 9 to 12)

- Full pipeline evaluation across all six source corpora, not just Buendia. [**Required**]
- Statistical comparison between fine-tuned TrOCR and the trained CRNN. [**Required**]
- Final documentation and a clean public-facing demo. [**Required**]

Timeline

Week	Work
1	Meet with mentors, plan ground truth expansion, annotate 5 to 10 pages for line detection
2 to 3	Train line detector, replace projection heuristic
4	Line-level CRNN training, check loss curves
5 to 6	Augmentation experiments, cross-attention CRNN variant
7	Midterm evaluation
8	Self-supervised pre-training experiments
9 to 10	Full evaluation across all 6 corpora

Week	Work
11	TrOCR vs CRNN comparison with statistical analysis
12	Final documentation, demo, cleanup

Related Work

Tesseract is the standard workhorse for historical OCR but needs careful layout pre-processing and often breaks on dense or ornate pages from this period. Kraken handles historical scripts better and is widely used in the digital humanities community, but it still requires line-level ground truth annotations to train on custom layouts. TrOCR is a strong zero-shot baseline because it was trained on a large mix of printed and handwritten text, but domain mismatch hits hard on 17th-century letterforms. During the evaluation test I ran `microsoft/trocr-base-printed` zero-shot on three pages from the Buendia Instruccion source and got a CER of about 1.15. The model was producing output like "WWW.LLM" and "GARDORIZIO ON 6020" where the ground truth is actual Spanish prose. That's not a calibration issue, it's a fundamental domain mismatch.

What makes this project different is the combination: a CRNN recognizer optimized specifically for the character distribution of period Spanish text, a lexicon decoder that biases toward historically plausible words at word boundaries, and an LLM post-correction step that can catch errors that are semantically obvious even when they're ambiguous visually. None of the existing tools use all three in combination.

Biographical Information

I hold an MSc in Computer Science (expected Dec 2026) from the University of New Haven—where my coursework has focused on AI, Computer Vision, and Algorithm Design—and a prior MSc in Information Technology from KNUST in Ghana. I also did my undergrad in Computer Science at KNUST.

I currently work as a Graduate Research Assistant at the SAIL Lab (University of New Haven), where I build NLP pipelines for processing unstructured PDF and email data using LLM APIs (OpenAI and Anthropic Claude), and architect full-stack dashboards in FastAPI and React to surface those results to stakeholders. Before that I was a Software Engineer at KNUST where I scaled data pipelines to 50M+ monthly data points, deployed a Django REST API serving 50K+ requests/day, and reduced query latency by 60% through database optimization.

I also did an internship at EXRX.NET where I shipped a mobile fitness app to iOS and Android in 12 weeks—15K users in the first month.

On the ML side, I've worked with PyTorch, NLP, and reinforcement learning. I built a Tower Defense AI using Q-Learning and PPO that achieved a 78% win rate versus a 62% A* baseline. I also built SentinelMD, an offline-capable clinical safety tool using MedGemma open-weight models, for Google's HAI-DEF hackathon.

For this application specifically, before submitting the proposal, I built a working end-to-end OCR pipeline: a CRNN from scratch in PyTorch with inverse-frequency character weighting, a pure-Python lexicon beam search decoder, Gemini API integration for post-correction with a rule-based fallback, TrOCR fine-tuning scripts using `Seq2SeqTrainer`, and a full CER/WER evaluation framework. Code is at https://github.com/Magma4/renaissance-ocr.

Baseline results on three matched evaluation pages:

Backend	CER	WER
TrOCR zero-shot raw	1.1468	1.3387
Rule-based cleanup	1.1423	1.3050
Gemini cleanup	1.1026	1.2879

The Gemini step caught genuine 17th-century OCR errors like capital I misread as lowercase l ("Ia" to "la").

Practical Notes

This is a 175-hour project. I'm available roughly 15 hours per week over the 12-week GSoC period. I develop on a MacBook with an M-series chip and run compute-heavy training on a cloud VM where I have GCP credits. I prefer async communication over email or chat and I'm happy to do weekly video syncs if the mentors prefer.